

MAMBA-Viper x86 Implementation Self-Evaluation Report

MAMBA-Viper Submission Team

June 28, 2026

Document Information

Algorithm name	MAMBA-Viper
Algorithm category	Public-key cryptography: Key Encapsulation Mechanism (KEM)
Target architecture	x86-64
Implementation versions	Reference implementation; performance-optimized implementation
Submission directory	API_PKC/

1 Scope of This Report

This report gives the x86 implementation self-evaluation of MAMBA-Viper. It covers two implementation versions.

- **Reference implementation.** The reference implementation is located under `API_PKC/Implementations/Reference_Implementation/`. It is written in ISO C99 and does not rely on AVX2 or other architecture-specific instruction sets.
- **Performance-optimized implementation.** The optimized implementation is located under `API_PKC/Implementations/Optimized_Implementation/`. It targets x86-64 processors with AVX2 support.

The evaluation covers the following five KEM instances.

Instance	Target	Public key	Secret key	Shared secret	Ciphertext
MAMBA-Viper-128	128	608	1424	16	736
MAMBA-Viper-192	192	992	2200	24	1088
MAMBA-Viper-256	256	1312	2912	32	1472
MAMBA-Viper-384	384	2496	5488	48	2656
MAMBA-Viper-512	512	3200	7040	64	3456

All sizes are given in bytes. All five instances set `VIPER_USE_E8_CODEC=1`.

2 Implementation Materials

2.1 Submission Directory Layout

The formal implementation submission boundary is:

API_PKC/

This directory is a self-contained API_PKC submission package. It contains the formal implementations, formal KAT outputs, and internal API conformance tests.

The main directory layout is as follows:

```
API_PKC/
  README
  Implementations/
    README
    Reference_Implementation/
      AlgorithmInstance/
        MAMBA-Viper-128/
        MAMBA-Viper-192/
        MAMBA-Viper-256/
        MAMBA-Viper-384/
        MAMBA-Viper-512/
      Optimized_Implementation/
        MAMBA-Viper-128/
        MAMBA-Viper-192/
        MAMBA-Viper-256/
        MAMBA-Viper-384/
        MAMBA-Viper-512/
    Test_Vectors/
      KAT_KEM_MAMBA-Viper-128.txt
      KAT_KEM_MAMBA-Viper-192.txt
      KAT_KEM_MAMBA-Viper-256.txt
      KAT_KEM_MAMBA-Viper-384.txt
      KAT_KEM_MAMBA-Viper-512.txt
    Internal_Tests/
      NGCC_API/
        api_check.c
```

The directory `Reference_Implementation/AlgorithmInstance/` is the retained official API_PKC template directory. Each `MAMBA-Viper-*` instance directory provides its own `KEM_AlgorithmInstance.h`, `KEM_AlgorithmInstance.c`, `Makefile`, official KAT generator, official DRNG files, official auxiliary-function files, Viper API compatibility layer, and the required Viper core source files.

2.2 Use of Official API_PKC Support Files

The following official API_PKC files are copied into each instance directory and remain byte-for-byte identical to the official template files:

- `KAT_KEM.c`
- `drng.c`
- `drng.h`
- `auxfunc.c`
- `auxfunc.h`

The following command was used to verify byte-for-byte identity:

```
for d in API_PKC/Implementations/*_Implementation/MAMBA-Viper-*; do
  for f in KAT_KEM.c drng.c drng.h auxfunc.c auxfunc.h; do
    cmp -s "API_PKC/Implementations/Reference_Implementation/AlgorithmInstance/$f" "$d/$f" || exit 1
  done
done
```

Result: **PASS.**

2.3 Randomness, Hash, and XOF Integration

In the API_PKC packaging, Viper’s randomness interface is adapted to the official DRNG through `viper_api_rng.c`. The compatibility symbol `randombytes` eventually calls:

```
get_random_number(&drng_algorithm, x, xlen * 8ULL)
```

The hash, XOF, and KDF compatibility symbols are implemented in `viper_api_hash.c`. The compatibility mapping is as follows:

- `shake128` and `shake256` are routed to `pseudoXOF`.
- `sha3_256` is routed to `sm3hash`, with `pseudoXOF` as fallback when needed.
- `sha3_512` is routed to `pseudohash`.

The API_PKC packaging path does not use `RAND_bytes`, OpenSSL, `/dev/urandom`, `getrandom`, or `arc4random`.

3 x86 Test Environment

3.1 Hardware Environment

CPU model	AMD Ryzen 7 9700X 8-Core Processor
Architecture	x86-64
CPU count	16 logical CPUs
CPU byte order	Little Endian
Instruction sets	SSE2, AES, PCLMULQDQ, AVX, AVX2, BMI1, BMI2, SHA-NI, AVX-512 extensions
Number of cores used	Single-core measurement
Cache information	L1d: 768 KiB; L1i: 512 KiB; L2: 16 MiB; L3: 256 MiB

3.2 Software Environment

Operating system	Ubuntu 22.04 series
Linux kernel	Linux 6.8.0-124-generic, x86-64
Compiler	GCC 11.4.0
Build tool	GNU Make 4.3
CMake	CMake 3.22.1
Timing method	Serialized x86 cycle counter used by <code>benchmark_kem_api.c</code>
Test date	2026-06-28T12:28:15Z
AVX2 availability	Yes
Result directory	<code>x86_Self_Evaluation/results/20260628-202815/</code>

4 Compilation Configuration

4.1 Reference Implementation

The reference implementation uses portable C99 code and does not require AVX2 or other architecture-specific instruction sets. The compilation flags are:

```
-std=c99 -Wpedantic -Wall -Wextra -O2
```

Example instance directory:

```
API_PKC/Implementations/Reference_Implementation/MAMBA-Viper-128/
```

The build commands are:

```
make clean
make
make kat
```

4.2 Performance-Optimized Implementation

The optimized implementation uses the x86-64 AVX2 path. The compilation flags are:

```
-O3 -march=x86-64 -mavx2 -mtune=native -flto -fomit-frame-pointer -std=c99 -Wpedantic -Wall -Wextra
```

Example instance directory:

```
API_PKC/Implementations/Optimized_Implementation/MAMBA-Viper-128/
```

The build commands are:

```
make clean
make
make kat
```

5 Functional Correctness Tests

5.1 Known-Answer Tests

The formal KAT files are located under:

```
API_PKC/Test_Vectors/
```

The files are:

```
KAT_KEM_MAMBA-Viper-128.txt
KAT_KEM_MAMBA-Viper-192.txt
KAT_KEM_MAMBA-Viper-256.txt
KAT_KEM_MAMBA-Viper-384.txt
KAT_KEM_MAMBA-Viper-512.txt
```

These files are generated by the unmodified official `KAT_KEM.c`. The KAT procedure covers key generation, encapsulation, and decapsulation for all five instances.

Instance	Reference KAT	Optimized KAT
MAMBA-Viper-128	PASS	PASS
MAMBA-Viper-192	PASS	PASS
MAMBA-Viper-256	PASS	PASS
MAMBA-Viper-384	PASS	PASS
MAMBA-Viper-512	PASS	PASS

The following commands were used to generate and check the KAT files:

```
for d in API_PKC/Implementations/Reference_Implementation/MAMBA-Viper-*; do
  (cd "$d" && make clean && make && make kat) || exit 1
done

for d in API_PKC/Implementations/Optimized_Implementation/MAMBA-Viper-*; do
  (cd "$d" && make clean && make && make kat) || exit 1
done

for lvl in 128 192 256 384 512; do
  cmp -s \
    "API_PKC/Test_Vectors/KAT_KEM_MAMBA-Viper-$lvl.txt" \
    "API_PKC/Implementations/Reference_Implementation/MAMBA-Viper-$lvl/output/
    KAT_KEM_MAMBA-Viper-$lvl.txt" || exit 1
done
```

Conclusion: all five Reference instances and all five Optimized instances pass the KAT tests.

5.2 API Conformance Tests

The internal API conformance test source file is:

```
API_PKC/Internal_Tests/NGCC_API/api_check.c
```

The test covers:

- kem_get_pk_len_bytes
- kem_get_sk_len_bytes
- kem_get_ss_len_bytes
- kem_get_ct_len_bytes
- output length writeback by kem_keygen
- output length writeback by kem_enc
- output length writeback by kem_dec
- implicit rejection for valid-length invalid ciphertexts
- negative error codes for invalid lengths and null pointers

Instance	Reference API check	Optimized API check
MAMBA-Viper-128	PASS	PASS
MAMBA-Viper-192	PASS	PASS
MAMBA-Viper-256	PASS	PASS
MAMBA-Viper-384	PASS	PASS
MAMBA-Viper-512	PASS	PASS

5.3 Implicit Rejection Test

Viper.KEM uses implicit rejection. For a valid-length ciphertext that fails the internal re-encryption check, `kem_dec` does not return an externally visible ciphertext-invalid error. Instead, it outputs a fallback shared secret and returns 0.

The test procedure is:

```

keygen
enc
copy ct to ct_bad
flip one byte of ct_bad
call kem_dec(sk, sk_len, ct_bad, ct_len, ss_bad, &ss_bad_len)

```

The expected behavior is:

- `kem_dec` returns 0.
- `ss_bad_len` equals the shared-secret length of the corresponding instance.
- Ciphertext validity is not exposed through the return value.

Conclusion: all five Reference instances and all five Optimized instances pass the implicit rejection test.

6 Performance Test Methodology

6.1 Tested Operations

The following KEM operations are evaluated:

- `kem_keygen`: key generation.
- `kem_enc`: encapsulation.
- `kem_dec`: decapsulation.

The benchmark uses the final `API_PKC` wrapper functions rather than the internal development API. The measured functions are `kem_keygen`, `kem_enc`, and `kem_dec`.

6.2 Number of Runs and Statistics

Each operation is measured independently for each algorithm instance and each implementation version.

Warm-up runs	100
Measurement runs	1000
Reported statistic	Median cycle count
Additional raw statistics	Mean, minimum, maximum, and standard deviation are recorded in <code>benchmark.csv</code>
Timing unit	CPU cycles

The benchmark records CPU cycle counts using a serialized x86 cycle counter. The benchmark also verifies shared-secret consistency during encapsulation and decapsulation. For invalid ciphertexts of valid length, the benchmark and API conformance tests verify the implicit rejection behavior.

7 Performance Results

The tables below report median cycle counts over 1000 measured runs after 100 warm-up runs. Median values are used as the main reported metric to reduce the effect of occasional operating-system scheduling noise.

7.1 Reference Implementation

Instance	KeyGen	Encaps	Decaps
MAMBA-Viper-128	157317	224195	269034
MAMBA-Viper-192	294114	391011	453711
MAMBA-Viper-256	477269	598107	676765
MAMBA-Viper-384	1433330	1934919	2094895
MAMBA-Viper-512	2611684	2881668	3043354

7.2 Performance-Optimized Implementation

Instance	KeyGen	Encaps	Decaps
MAMBA-Viper-128	118558	166436	184296
MAMBA-Viper-192	202915	269414	286324
MAMBA-Viper-256	303994	378472	394051
MAMBA-Viper-384	811473	929461	950739
MAMBA-Viper-512	1320662	1517117	1544097

7.3 Optimized-over-Reference Speedup

The following table reports the speedup of the optimized implementation over the reference implementation, computed from the median cycle counts.

Instance	KeyGen	Encaps	Decaps
MAMBA-Viper-128	1.33x	1.35x	1.46x
MAMBA-Viper-192	1.45x	1.45x	1.58x
MAMBA-Viper-256	1.57x	1.58x	1.72x
MAMBA-Viper-384	1.77x	2.08x	2.20x
MAMBA-Viper-512	1.98x	1.90x	1.97x

8 Resource Consumption

8.1 Communication and Storage Sizes

Instance	Public key	Secret key	Shared secret	Ciphertext
MAMBA-Viper-128	608	1424	16	736
MAMBA-Viper-192	992	2200	24	1088
MAMBA-Viper-256	1312	2912	32	1472
MAMBA-Viper-384	2496	5488	48	2656
MAMBA-Viper-512	3200	7040	64	3456

All sizes are identical for the Reference and Optimized implementations because both implementations expose the same API_PKC instance parameters and serialization formats.

8.2 Static Memory, Peak Memory, and Stack Usage

This run records communication and storage sizes, build status, KAT status, API conformance status, dependency-scan results, artifact-scan results, and cycle-count benchmark results. Static memory, peak resident memory, and maximum stack usage were not measured in this run. They can be added by extending the self-evaluation harness with `/usr/bin/time -v`, compiler stack-usage output, or an equivalent resource-measurement tool.

9 Dependency Description

Compiler	GCC 11.4.0.
Build tool	GNU Make 4.3.
CMake	CMake 3.22.1.
Randomness	API_PKC drng.c/h, used for KAT generation, API testing, and deterministic benchmark initialization.
Hash/XOF	API_PKC auxfunc.c/h, providing sm3hash, pseudohash, and pseudoXOF.
Third-party cryptographic libraries	None. The API_PKC packaging path does not depend on OpenSSL.
System randomness	None. The API_PKC packaging path does not use /dev/urandom, getrandom, or arc4random.

The forbidden-dependency scan command is:

```
rg -n "RAND_bytes|OpenSSL|/dev/urandom|getrandom|arc4random" \  
API_PKC/Implementations/Reference_Implementation/MAMBA-Viper-* \  
API_PKC/Implementations/Optimized_Implementation/MAMBA-Viper-*
```

Result: no match.

10 Reproducibility

10.1 Self-Evaluation Harness

The self-evaluation harness is located under:

```
x86_Self_Evaluation/
```

The main one-click script is:

```
x86_Self_Evaluation/run_api_pkc_self_eval.sh
```

The benchmark source is:

```
x86_Self_Evaluation/benchmark_kem_api.c
```

The benchmark measures the final API_PKC wrapper functions and does not directly call the internal development API.

10.2 Reproduction Commands

The formal run reported in this document used:

```
bash x86_Self_Evaluation/run_api_pkc_self_eval.sh --runs 1000 --warmup 100
```

The result directory is:

```
x86_Self_Evaluation/results/20260628-202815/
```

After extracting the submission package, the build and KAT checks can also be reproduced with:

```

for d in API_PKC/Implementations/Reference_Implementation/MAMBA-Viper-*; do
  (cd "$d" && make clean && make && make kat) || exit 1
done

for d in API_PKC/Implementations/Optimized_Implementation/MAMBA-Viper-*; do
  (cd "$d" && make clean && make && make kat) || exit 1
done

```

KAT consistency check:

```

for lvl in 128 192 256 384 512; do
  cmp -s \
    "API_PKC/Test_Vectors/KAT_KEM_MAMBA-Viper-$lvl.txt" \
    "API_PKC/Implementations/Reference_Implementation/MAMBA-Viper-$lvl/output/
    KAT_KEM_MAMBA-Viper-$lvl.txt" || exit 1
done

```

Build-artifact cleanup check:

```

find API_PKC -type f \( -name "*.o" -o -name "*.a" -o -name "*.so" -o -name "*.pdb" -o
  -name "KAT_KEM" -o -name "api_check" -o -name "KAT_KEM_API_PKC" -o -name "
  benchmark_kem_api" \) -print

```

Expected result: no output.

10.3 Raw Logs

The raw logs of this self-evaluation are stored under:

```
x86_Self_Evaluation/results/20260628-202815/
```

The main generated evidence files are:

- environment.txt
- git_status.txt
- build_kat_reference.log
- build_kat_optimized.log
- kat_consistency.log
- api_check.log
- benchmark_reference.log
- benchmark_optimized.log
- benchmark.csv
- sizes.csv
- dependency_scan.log
- artifact_scan.log
- package_check.log
- summary.md

11 Self-Evaluation Conclusion

Based on the tests described in this report:

- The MAMBA-Viper API_PKC package covers five KEM instances: 128, 192, 256, 384, and 512.
- Both the Reference implementation and the performance-optimized implementation can be built independently.
- All five instances can generate KAT files using the unmodified official `KAT_KEM.c`.
- The formal KAT files are stored under `API_PKC/Test_Vectors/`.
- The API wrapper correctly implements length queries, key generation, encapsulation, and decapsulation.
- `kem_dec` preserves implicit rejection: valid-length invalid ciphertexts do not expose ciphertext validity through the return value.
- The API_PKC packaging path uses the official DRNG and official auxiliary functions.
- The API_PKC packaging path does not depend on OpenSSL or system randomness.
- The dependency scan reports no forbidden dependency matches.
- The build-artifact scan reports no uncleaned build artifacts in `API_PKC/`.
- On AMD Ryzen 7 9700X, the optimized implementation improves median cycle counts over the reference implementation by approximately 1.33x–1.98x for key generation, 1.35x–2.08x for encapsulation, and 1.46x–2.20x for decapsulation across the five instances.

Final conclusion:

PASS. Under the test environment and methodology described in this report, the MAMBA-Viper x86 Reference implementation and x86 AVX2 performance-optimized implementation pass functional correctness tests, API conformance tests, KAT generation, size checks, dependency checks, artifact-cleanliness checks, and API-level benchmark execution. The performance results are reported as median cycle counts over 1000 measured runs after 100 warm-up runs.

A Appendix A: Command Summary

A.1 One-Click Self-Evaluation

```
bash x86_Self_Evaluation/run_api_pkc_self_eval.sh --runs 1000 --warmup 100
```

A.2 Reference Build and KAT

```
for d in API_PKC/Implementations/Reference_Implementation/MAMBA-Viper-*; do
  echo "Testing $d"
  (cd "$d" && make clean && make && make kat) || exit 1
done
```

A.3 Optimized Build and KAT

```
for d in API_PKC/Implementations/Optimized_Implementation/MAMBA-Viper-*; do
  echo "Testing $d"
  (cd "$d" && make clean && make && make kat) || exit 1
done
```

A.4 Formal KAT Consistency Check

```
for lvl in 128 192 256 384 512; do
  cmp -s \
    "API_PKC/Test_Vectors/KAT_KEM_MAMBA-Viper-$lvl.txt" \
    "API_PKC/Implementations/Reference_Implementation/MAMBA-Viper-$lvl/output/
    KAT_KEM_MAMBA-Viper-$lvl.txt" || exit 1
done
```

A.5 Forbidden Dependency Scan

```
rg -n "RAND_bytes|OpenSSL|/dev/urandom|getrandom|arc4random" \
  API_PKC/Implementations/Reference_Implementation/MAMBA-Viper-* \
  API_PKC/Implementations/Optimized_Implementation/MAMBA-Viper-*
```

A.6 Build-Artifact Scan

```
find API_PKC -type f \( -name "*.o" -o -name "*.a" -o -name "*.so" -o -name "*.pdb" -o
  -name "KAT_KEM" -o -name "api_check" -o -name "KAT_KEM_API_PKC" -o -name "
  benchmark_kem_api" \) -print
```

B Appendix B: Raw Benchmark Data

The raw benchmark data are stored in:

```
x86_Self_Evaluation/results/20260628-202815/benchmark.csv
```

For each implementation, instance, and KEM operation, the CSV file records the timestamp, implementation type, instance name, security level, operation, warm-up runs, measurement runs, mean cycles, median cycles, minimum cycles, maximum cycles, standard deviation, object sizes, status, and notes.

C Appendix C: Raw Environment and Test Logs

The raw environment and test logs are stored in:

```
x86_Self_Evaluation/results/20260628-202815/
```

The run summary reports:

```
Runs: 1000
Warmup: 100
AVX2: yes
Status: PASS
```